

Flower Power Project

Documentation

Alexandra Blaga
Ovidiu Alexandru Lates
Monika Mezei
Bogdan Marcel Somesan
Carmina-Paula Pitar

Table of contents:

1. Project Overview	2
1.1. Purpose	3
1.2. Scope	3
1.3. Technology Stack.....	3
2. Requirements	3
2.1. Functional Requirements	3
2.2. Non-Functional Requirements.....	4
3. Domain Model	6
3.1. Board Game Model	6
3.2. User Preferences	6
3.3. Database Diagram	7
4. System Architecture	7
4.1. Architecture Diagram	8
4.2. API Layer.....	8
4.3. Service Layer	9
4.4. Persistence Layer	9
4.5. AI Integration	9
5. System Design	9
5.1. Use-Case Diagram.....	10
5.2 Sequence Diagrams.....	10
6. API Design	12
7. Development Conventions.....	13
7.1. Branch Naming Convention.....	13
8. Initial Setup.....	15
8.1 Clone the repository	15
8.2 Start Rancher Desktop.	15
8.3 Create the local .env file.....	16

8.4 Start SQL Server and SonarQube.....	16
8.5 Configure the application's User Secrets	17
8.6 Restore .NET dependencies.....	17
8.7 Restore .NET tools	18
8.8 Create/update the application database	18
8.9 Optional — verify the database with SSMS	19
8.10 Run the application	19
8.11 Access SonarQube	20
8.12 Run SonarQube analysis	20
8.13 Normal development after the initial setup	21
8.14 Stopping the local environment.....	21
9 Definition of Ready and Done.....	22
9.1 Definition of Ready.....	22
9.2 Definition of Done.....	23

1. Project Overview

1.1. Purpose

Flower Power Project is a web-based board game discovery and recommendation platform. Users can browse games, manage personal preferences and favourites, rate games, and receive AI-powered recommendations based on their interests and requirements.

1.2. Scope

The application focuses exclusively on board games and is developed as a full-stack web application using a layered architecture. It includes users and administrator functionality, persistent storage, authentication and authorization, and AI-assisted recommendations.

1.3. Technology Stack

Frontend	-
Backend	.NET 10 Web API
Database	SQL Server
ORM	Entity Framework Core
Authentication	JWT
Testing	xUnit
Code Quality	SonarQube
Infrastructure	Docker Compose, Rancher Desktop
Source Control	Git, GitHub
AI Integration	OpenAI

2. Requirements

2.1. Functional Requirements

Authentication and User Management:

- **User Registration** - The system shall allow users to create a new account.
- **User Login** - The system shall allow registered users to log in using their credentials.
- **User Profile Management** - The system shall allow authenticated users to view and update their personal profile.

- **User management** - The system shall allow administrators to create, view, update, and delete users.

Board games:

- **Browse Board Games** - The system shall allow users to browse the available board games.
- **Board Game Management** - The system shall allow administrators to create, view, update, and delete board games.

Preferences:

- **Manage Preferences** – The system shall allow authenticated users to create, view, update, and delete their board game preferences.
- **Define Recommendation Criteria** – The system shall allow users to define preferences based on supported board game characteristics.

Favourites:

- **Manage Favourites** – The system shall allow authenticated users to manage their favourites.

Ratings:

- **Rate Board Games** - The system shall allow authenticated users to rate board games.

AI Recommendations:

- **Request AI Recommendations** – The system shall allow authenticated users to request AI-powered board game recommendations.
- **Personalized Recommendations** – The system shall generate recommendations based on the user's defined preferences.
- **Favourite-Based Recommendations** – The system shall be able to recommend games like the games saved in the user's favourites.
- **View Recommendations** – The system shall allow the users to view the recommendations generated for them.

Administration:

- **Category Management** – The system shall allow administrators to manage board games categories.
- **AI Usage Monitoring** – The system shall allow administrators to monitor AI usage.

2.2. Non-Functional Requirements

Security

- Secure password storage using hashing.
- JWT-based authentication.
- Role-based authorization.
- Secure refresh-token/session handling.
- Users must only access their own profile, preferences, favorites, and ratings.
- Admin endpoints must be restricted to administrators.

Performance

- Fast response times for browsing games and loading user data.
- Efficient database queries.
- Pagination for large game lists.
- AI requests should have timeout and error-handling mechanisms.

Scalability

- The system should support increasing numbers of users and games.
- The architecture should allow additional game types and genres.
- The recommendation feature should be extendable without redesigning the whole application.

Maintainability

- Clean Architecture.
- SOLID principles.
- DRY principle.
- KISS principle.
- YAGNI principle.
- Separation between controllers, services, repositories, DTOs, and entities.
- Clear API documentation using Swagger/OpenAPI.

Reliability

- Proper validation of user input.
- Consistent error responses.
- Database constraints to protect data integrity.
- Handling of failed AI or database requests.
- No loss or corruption of user preferences, favorites, or ratings.

Usability

- Good user experience.
- Clear navigation.

- Easy-to-use login, profile, preferences, favorites, and recommendation pages.

Quality assurance

- Unit tests for services and business logic.
- Tests for the recommendation engine.
- Minimum 70% code coverage.
- SonarQube analysis for:
 - o code smells;
 - o duplications;
 - o maintainability;
 - o test coverage.

Documentation and deployment

- Swagger/OpenAPI API documentation.
- Setup instructions.
- Architecture and database documentation.
- Git-based source control.
- Optional Docker and CI/CD support.

3. Domain Model

3.1. Board Game Model

Board games represent the main domain of the application. Each game contains general information used for displaying and managing the game, as well as characteristics that can be used when recommending games based on the user's preferences.

Property	Description
Name	Name of the board game
Type	Types of games, such as board, dices or cards, etc.
Price	Approximate price of the game
MinPlayers	Minimum supported number of Players
MaxPlayers	Maximum supported number of players
MinimumAge	Minimum recommended player age
PlayTime	Approximate duration of a game
LearningTime	Approximate time required to learn the game
Gerne	Game category/gerne

3.2. User Preferences

User preferences describe the characteristics a user generally looks for in a board game and are used as one of the inputs for generating personalized recommendations. Preferences are based on relevant board games characteristics such as budget, type, genre, budget, play time, number of players, age group.

3.3. Database Diagram

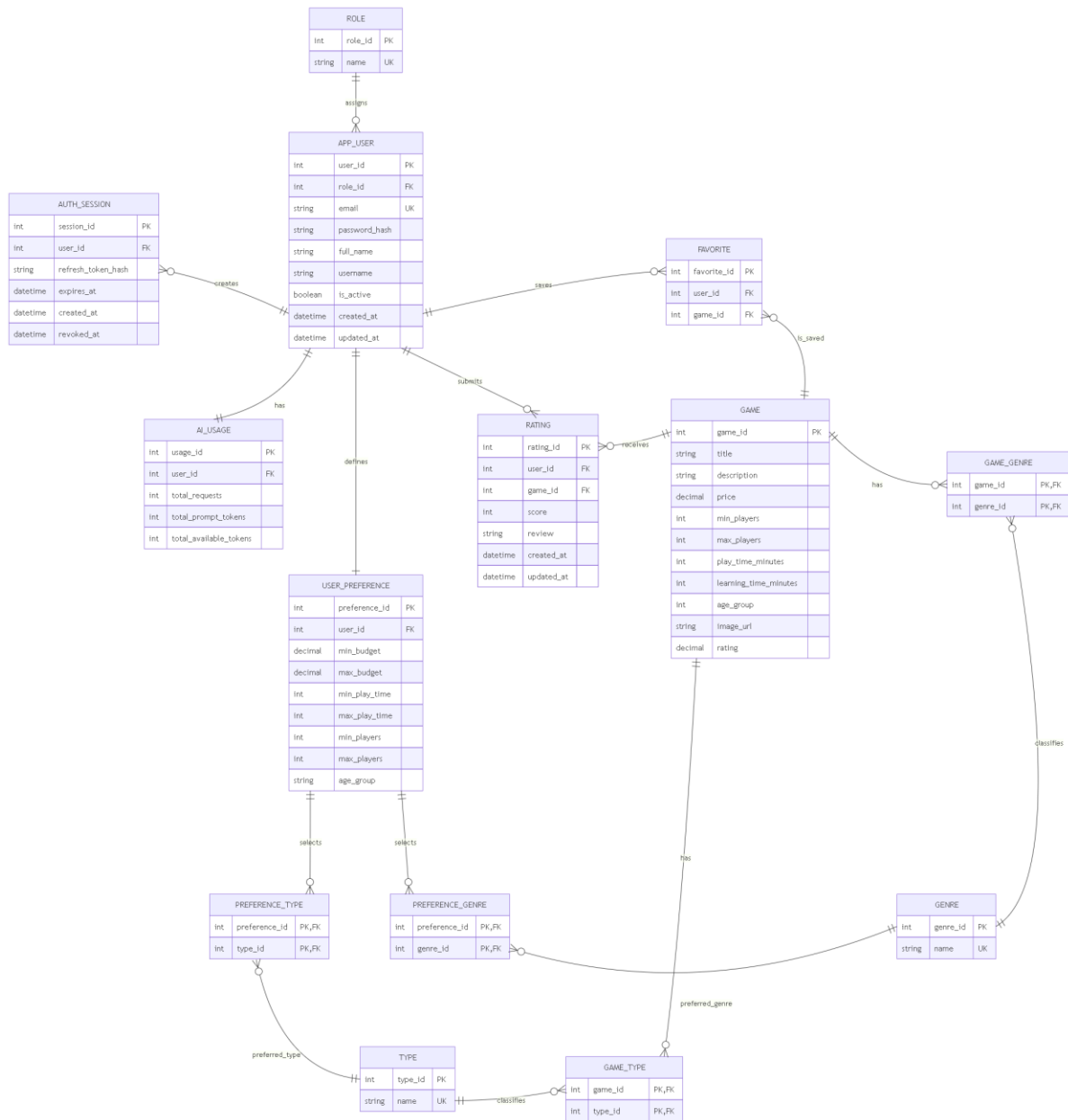


Figure 1 - Database Diagram

4. System Architecture

The application uses a layered architecture to provide separation of concerns while remaining appropriate for the size and expertise of the development team and the timeframe of the project. Each layer has a different responsibility, with dependencies flowing from the API layer through the business logic towards persistence and external integrations.

4.1. Architecture Diagram

The following diagram presents the main layers of the application and the interactions between their components

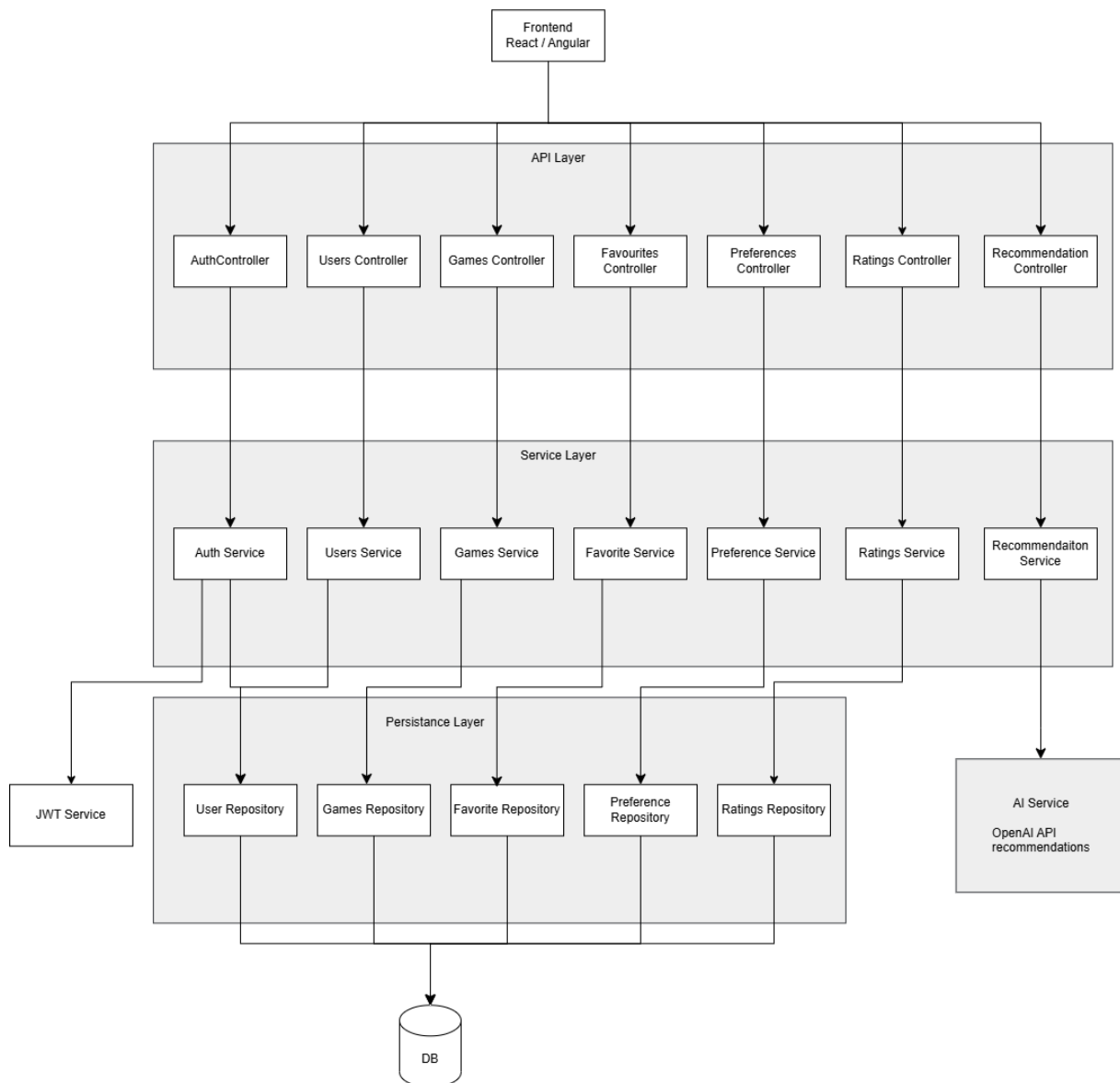


Figure 2 - Architecture Diagram

4.2. API Layer

The API layer represents the entry point to the backend application. Controllers define the available HTTP endpoints, receive requests from the frontend and delegate the required operations to the corresponding services. This layer is also responsible for returning the appropriate HTTP responses to the client.

4.3. Service Layer

The service layer contains the main business logic for the application. Services coordinate the operations required by each feature and use repositories or external integrations when additional data or functionality is required. Keeping the business logic outside the controllers also allows it to be tested independently.

4.4. Persistence Layer

The persistence layer is responsible for storing and retrieving application data. Repositories provide the service layer with access to the required data without exposing database-specific operations to the business logic. Entity Framework Core is used to interact with SQL Server.

4.5. AI Integration

AI integration is kept separate from the recommendation business logic. The RecommendationService determines what information is required for a recommendation, while the AI integration component is responsible for communicating with the external AI service and returning its response.

5. System Design

5.1. Use-Case Diagram

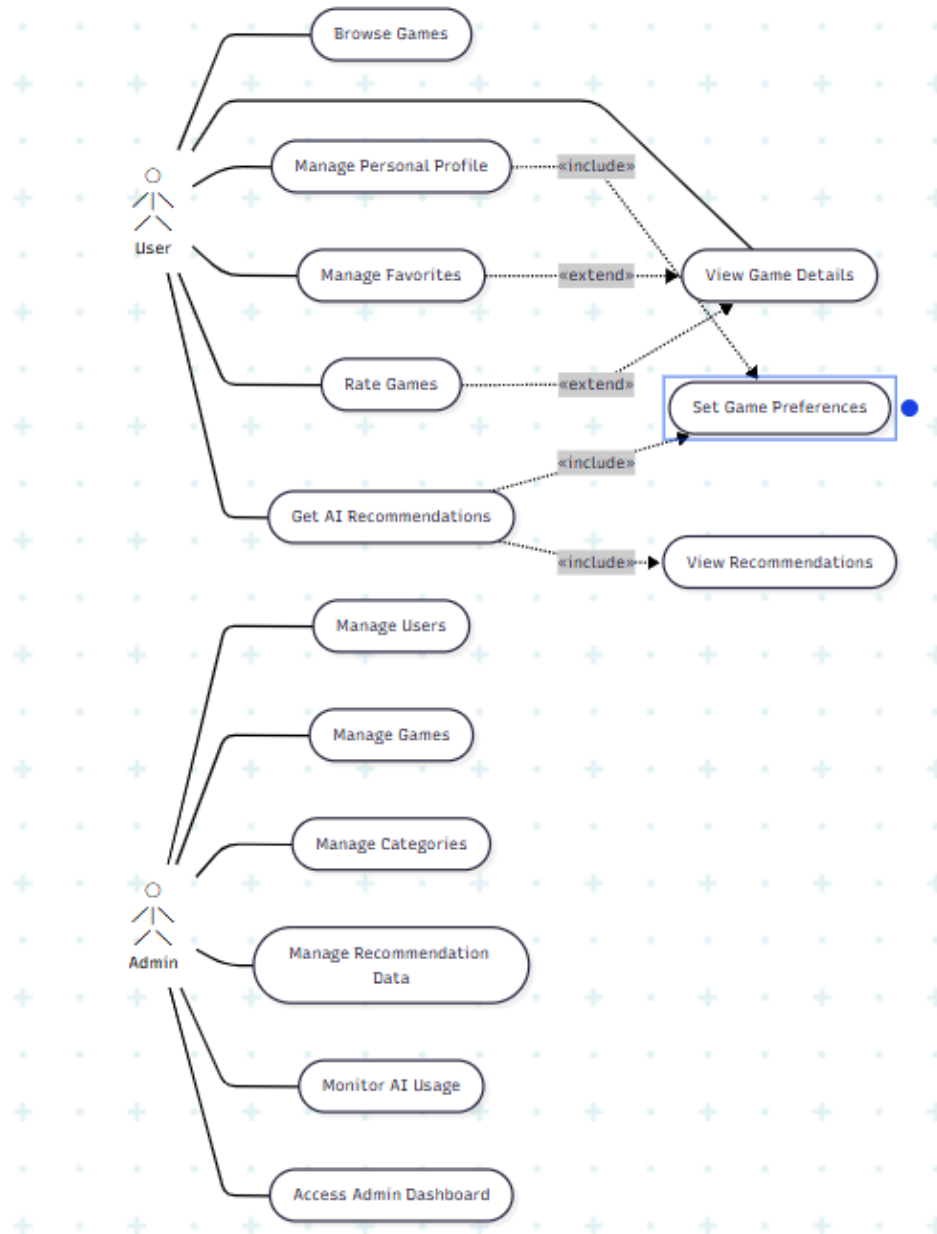


Figure 3 - Use Case Diagram

5.2 Sequence Diagrams

The following sequence diagram illustrates the **login process**, from the user submitting their credentials to the authentication of the user and generation of the JWT token.

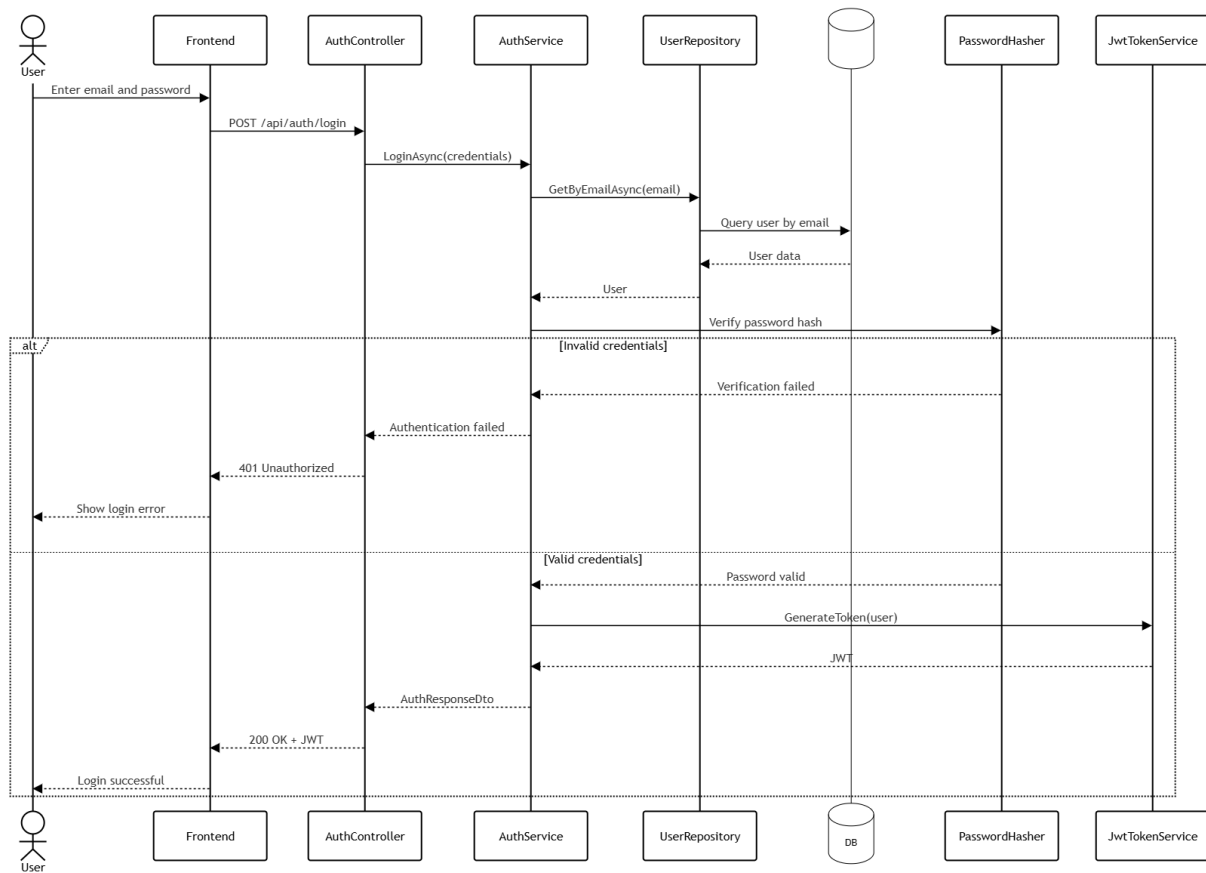


Figure 4 - Login Sequence Diagram

The following sequence diagram illustrates the **game recommendation process**, from the user requesting recommendations to the retrieval of their preferences, favorites, and ratings, followed by the generation and display of personalized game recommendations.

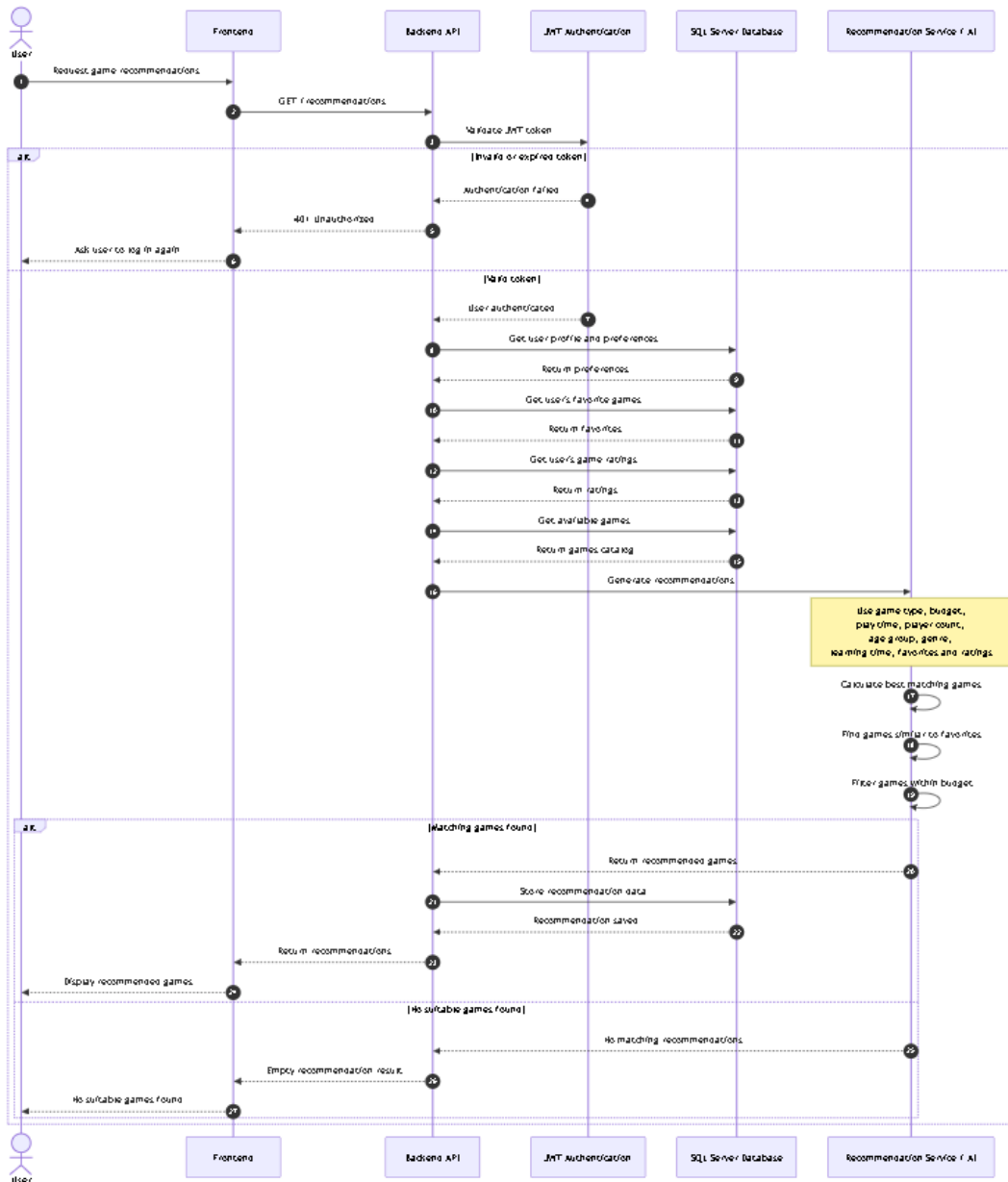


Figure 5 - Game Recommendation Sequence Diagram

6. API Design

Endpoint overview

Method	Path	Purpose	JWT
POST	/api/auth/register	Register a user	No
POST	/api/auth/login	Authenticate a user	No
GET	/api/users	List users	Yes
GET	/api/users/{userId}	Get a user	Yes
PUT	/api/users/{userId}	Update a user	Yes
DELETE	/api/users/{userId}	Delete a user	Yes
GET	/api/games	Browse games	No
POST	/api/games	Create a game	Yes
GET	/api/games/{gameId}	Get a game	No
PUT	/api/games/{gameId}	Update a game	Yes
DELETE	/api/games/{gameId}	Delete a game	Yes
GET	/api/preferences	Get current user preferences	Yes
POST	/api/preferences	Create current user preferences	Yes
PUT	/api/preferences	Replace current user preferences	Yes
DELETE	/api/preferences	Delete current user preferences	Yes
GET	/api/favorites	List current user's favorite games	Yes
PUT	/api/favorites/{gameId}	Create or replace a favorite	Yes
POST	/api/favorites/{gameId}	Add a game to favorites	Yes
DELETE	/api/favorites/{gameId}	Remove a game from favorites	Yes
PUT	/api/ratings/{gameId}	Create or replace the current user's rating	Yes
POST	/api/recommendations	Generate recommendations from preferences and favorites	Yes
GET	/api/admin/usage	Monitor AI usage	Yes
GET	/api/admin/categories	List game types and genres	No
POST	/api/admin/categories	Create a game type or genre	Yes

7. Development Conventions

To maintain a consistent workflow across the team, a set of shared conventions is used for code changes and project organization.

7.1. Branch Naming Convention

All development branches should follow a consistent naming convention to make it easy to identify the **owner of the branch**, the **associated work item**, and the **purpose of the change**.

Format

<username>/<ticket-id>-<short-description>

Examples

```
andrei/PROJ-123-add-user-filter
andrei/PROJ-145-fix-date-validation
maria/PROJ-152-update-dashboard
john/PROJ-160-add-api-validation
```

Rules

1. **Start with your username or agreed team identifier**
 - a. This identifies the developer primarily responsible for the branch.
 - b. Use the same identifier consistently across branches.
2. **Include the ticket/work-item ID**
 - a. Use the ID from the team's issue-tracking system (e.g. Jira or Azure DevOps).
 - b. Keep the ticket ID in its standard format, such as PROJ-123.
3. **Add a short description**
 - a. Briefly describe the purpose of the branch.
 - b. Use lowercase words separated by hyphens (-).
 - c. Keep the description concise and meaningful.
4. **Do not use spaces or special characters**
 - a. Use hyphens to separate words.
 - b. Avoid underscores where possible.
5. **Keep branch names concise**
 - a. The branch name should provide enough information to understand its purpose without becoming unnecessarily long.

Good

```
andrei/PROJ-123-add-user-filter
maria/PROJ-152-update-dashboard
john/PROJ-160-fix-api-validation
```

Avoid

```
PROJ-123
andrei/my-changes
andrei/PROJ-123-add-some-new-changes-to-the-user-filter-component
andrei/PROJ-123_add_user_filter
```

Shared Branches

The username prefix is intended for branches primarily owned by one developer. For branches where multiple developers are actively collaborating, the username may be omitted, and an agreed team or branch type can be used instead.

```
team/PROJ-200-payment-integration
```

The most important requirement is that the convention is applied consistently across the repository.

8. Initial Setup

8.1 Clone the repository

Open PowerShell and run:

```
git clone https://github.com/ovidiu-lates/flower-power-project.git
cd flower-power-project
```

The repository already contains the shared project configuration, including:

```
compose.yml
dotnet-tools.json
games.slnx
games/
```

Do not manually create the SQL Server or SonarQube containers. Docker Compose manages them.

8.2 Start Rancher Desktop.

Make sure Rancher Desktop is configured with:

```
Container Engine: dockerd (moby)
Kubernetes: Disabled
```

Verify that Docker is available by running:

```
docker version
```


You can also verify Compose:

```
docker compose version
```

Both commands should complete successfully before continuing.

8.3 Create the local .env file

The SQL Server administrator password must not be committed to GitHub.

The repository contains:

```
.env.example
```

Create your own .env file from it:

```
Copy-Item .env.example .env
```

Open .env and replace the placeholder with your own strong local password:

```
MSSQL_SA_PASSWORD=<THEIR_LOCAL_PASSWORD>
```

For example:

```
MSSQL_SA_PASSWORD=MyLocalPassword123!
```

The SQL Server password must satisfy SQL Server's password requirements.

.env is ignored by Git and must never be committed.

8.4 Start SQL Server and SonarQube

The repository's compose.yml contains the configuration for both SQL Server and SonarQube.

Start all local infrastructure with:

```
docker compose up -d
```

Docker Compose creates the required containers and persistent volumes automatically.

Verify the containers:

```
docker compose ps
```

You should see the SQL Server and SonarQube services running.

If SQL Server is still starting, check:

```
docker compose logs -f sqlserver
```

Press Ctrl+C to stop following the logs. This does not stop the container.

8.5 Configure the application's User Secrets

The .NET application runs locally from Visual Studio, while SQL Server runs inside Rancher Desktop.

Open the project in Visual Studio.

Right-click the API/startup project and select:

Manage User Secrets

Configure:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost,1433;Database=FlowerPowerGamesDb;User
Id=sa;Password=<THEIR_LOCAL_PASSWORD>;Encrypt=False;"
  }
}
```

Replace:

```
<THEIR_LOCAL_PASSWORD>
```

with the same password configured in .env.

For example, if .env contains:

```
MSSQL_SA_PASSWORD=MyLocalPassword123!
```

then User Secrets should contain:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost,1433;Database=FlowerPowerGamesDb;User
Id=sa;Password=MyLocalPassword123!;Encrypt=False;"
  }
}
```

User Secrets are local to the developer's machine and must not be committed to GitHub.

8.6 Restore .NET dependencies

From the repository root, run:

```
dotnet restore
```

This restores the project's NuGet dependencies.

8.7 Restore .NET tools

The repository contains the .NET tool configuration required by the project, including SonarScanner.

Run:

```
dotnet tool restore
```

This means developers do not need to globally install SonarScanner with:

```
dotnet tool install --global dotnet-sonarscanner
```

The project's required tool versions are defined in Git and restored locally.

8.8 Create/update the application database

You do not need to manually open SSMS and run:

```
CREATE DATABASE FlowerPowerGamesDb;
```

The database schema is managed through the Entity Framework Core migrations committed to the repository.

From Visual Studio

Open:

Tools → NuGet Package Manager → Package Manager Console

Make sure the correct project containing AppDbContext is selected as the Default Project.

Run:

```
Update-Database
```

Alternatively, from normal PowerShell

Do not use Update-Database directly in normal PowerShell.

Instead use:

```
dotnet ef database update
```

After the migration completes, FlowerPowerGamesDb and its schema should be available in the SQL Server container.

8.9 Optional — verify the database with SSMS

SSMS is not required just to create the database anymore, but it can be used to inspect or debug it.

Connect using:

```
Server: localhost,1433
Authentication: SQL Server Authentication
Username: sa
Password: <THEIR_LOCAL_PASSWORD>
```

You should see:

```
Databases
└─ FlowerPowerGamesDb
```

with the tables created by the EF Core migrations.

8.10 Run the application

Open:

```
games.slnx
```

in Visual Studio.

Select the API project as the startup project if necessary.

Press: F5

The application should start and connect to FlowerPowerGamesDb running inside the SQL Server container.

8.11 Access SonarQube

SonarQube is started automatically by:

```
docker compose up -d
```

Open:

<http://localhost:9000>

For a fresh local SonarQube installation, the initial credentials are:

```
Username: admin
Password: admin
```

SonarQube will require the developer to change the password.

Each developer has their own local SonarQube instance. Developers should not share SonarQube administrator passwords or analysis tokens.

Create the local project using:

Create a local project

```
Use:
Project name: Flower Power Games
Project key: flower-power-games
```

Select: Locally as the analysis method.

Generate a local analysis token when requested.

Never commit SonarQube tokens to GitHub.

8.12 Run SonarQube analysis

Because the SonarScanner tool is configured as a repository-local .NET tool, first ensure tools have been restored:

```
dotnet tool restore
```

Then use the SonarScanner commands provided by the local SonarQube instance.

The general process is:

```
SonarScanner begin
↓
dotnet build
↓
dotnet test / coverage
↓
SonarScanner end
↓
http://localhost:9000
```

Each developer should use their own local SonarQube token.

The generated scanner directory:

```
.sonarqube/
```

is a local build/analysis artifact and must remain excluded through .gitignore.

8.13 Normal development after the initial setup

The steps above are mostly one-time setup.

After a developer has configured everything once, their normal workflow is much shorter.

Start Rancher Desktop and run:

```
git pull
docker compose up -d
dotnet restore
dotnet tool restore
dotnet ef database update
```

Then open Visual Studio and press:

F5

dotnet ef database update is safe to run when there are no new migrations; EF Core determines which migrations have already been applied.

8.14 Stopping the local environment

To stop the containers:

```
docker compose stop
```

To stop and remove the containers:

```
docker compose down
```

The persistent volumes remain, so the SQL Server database and SonarQube data are retained.

Later, start everything again with:

```
docker compose up -d
```

Important: do not normally use

```
docker compose down -v
```

The -v option removes the Compose volumes and can permanently delete the developer's local SQL Server database and local SonarQube data.

9 Definition of Ready and Done

9.1 Definition of Ready

A work item is considered **Ready** when:

- The objective and expected outcome are clearly defined.
- The description provides enough information for the team to understand the requirements.
- Acceptance criteria are defined, clear, and testable.
- Dependencies are identified and, where necessary, resolved or planned.
- Required designs, specifications, API contracts, or supporting documentation are available.
- Technical questions or major uncertainties have been clarified.
- The work item has been reviewed and understood by the team.

- The work item is small enough to be completed within the planned sprint.
- There are no known blockers preventing the team from starting the work.

9.2 Definition of Done

A work item is considered **Done** when:

- All acceptance criteria have been met.
- Implementation is complete and follows agreed coding and architecture standards.
- Code has been peer-reviewed and approved.
- Automated tests have been added or updated where applicable and are passing.
- Required functional, integration, and regression testing has been completed successfully.
- No critical or high-priority defects related to the work item remain unresolved.
- Security, performance, accessibility, and other non-functional requirements have been addressed where applicable.
- Required documentation has been created or updated.
- The implementation has been successfully integrated into the target branch/environment.
- The completed functionality has been validated against the original requirements.
- The work item has been reviewed/accepted by the appropriate stakeholder where required.